

On-Policy vs. Off-Policy Deep Reinforcement Learning for Continuous Control in Autonomous Racing

Team 7

F1TENTH Autonomous Racing Course Project

Abstract—We present a study of on-policy and off-policy deep reinforcement learning for end-to-end LiDAR-based control on the F1TENTH autonomous racing platform. Specifically, we train Proximal Policy Optimization (PPO) and Soft Actor-Critic (SAC) agents that map raw 2D LiDAR observations directly to continuous steering commands, and we evaluate both against a Pure Pursuit baseline. We document the iterative reward-shaping and observation-space choices required to make each algorithm converge, the use of a short-segment curriculum to handle long tracks, and a controlled sim-to-sim study quantifying the cost of the kinematic-vs.-dynamic simulator gap. PPO learns to follow straights and most corners on the Spielberg track from random spawn points, reaching a mean progress gain of 20.6% on full-track evaluation. On the SAC side, a Spielberg-focused curriculum reaches a best validation clean completion rate of 1.00 on a 25 m segment at 60k training steps and 1.00 on a 40 m segment at 80k training steps. A broader 10-episode reevaluation of the best 40 m checkpoint gives a 0.40 segment completion rate under matched conditions. Under a matched 40 m segment evaluation, SAC achieves 71.7% mean progress and 9.37 mean reward, compared with PPO’s 65.2% mean progress and 8.9 mean reward. These results suggest that PPO is more constrained by reward design and observation simplification, whereas SAC benefits more from curriculum scaling but remains sensitive to simulator dynamics.

Index Terms—reinforcement learning, autonomous racing, F1TENTH, PPO, SAC, sim-to-sim transfer

I. INTRODUCTION

A. The Problem

F1TENTH is widely used in research and education. Each car is equipped with a 2D LiDAR scanner and runs ROS 2, mirroring the sensor and software stack used in much of the full-scale autonomous-vehicle industry. The task we tackle is end-to-end *continuous control*: from raw LiDAR (and minimal proprioception) the agent must produce continuous steering commands that drives the car through a track.

Classical control pipelines for F1TENTH (e.g. Pure Pursuit on a tuned waypoint file, or model-predictive control on a known centerline) already drive the platform competently, but they are brittle to parameter changes, depend on a global pose estimate, and require hand-engineered reference trajectories. Deep reinforcement learning (RL) is an attractive end-to-end alternative because it learns the mapping from sensor to action directly, optimises the lap-level objective, and in principle generalises to new tracks without re-tuning. The open question we focus on is which family of RL algorithms is the right choice for this problem.

The two most popular candidates in the continuous-control literature are: **Proximal Policy Optimization (PPO)** [1], an on-policy actor-critic method, and **Soft Actor-Critic (SAC)** [2], [3], an off-policy maximum-entropy method. Despite both algorithms being applied to F1TENTH-style problems individually, no clean head-to-head benchmark on the F1TENTH platform exists in the public literature. This report investigates that gap, with two complementary studies on PPO and SAC respectively, and a comparative study between the two algorithms’ performance on the Spielberg track.

B. What Others Have Tried

Prior work on F1TENTH and on small autonomous racing platforms has explored a wide spectrum of approaches:

- **Classical control baselines.** Pure Pursuit and MPC remain the de-facto baselines on F1TENTH. They drive reliably given a tuned waypoint file and an exact pose estimate, but assume privileged inputs not available end-to-end.
- **End-to-end deep RL.** Several works train PPO or SAC on LiDAR observations [7]. Reported results vary strongly with reward shaping, simulator fidelity, and curriculum choices; head-to-head comparisons on the same platform are scarce.
- **Imitation and residual learning.** Other lines of work imitate classical controllers or learn residuals on top of MPC. These typically achieve high completion rates but inherit the assumptions of their teacher.
- **Sim-to-real / sim-to-sim transfer.** Domain randomisation and high-fidelity simulators (e.g. `f1t10_gym`) are used to narrow the dynamics gap, but the cost of *not* doing so has rarely been quantified directly on the F1TENTH stack.

C. Our Approach in Brief

We implement and train both PPO and SAC for LiDAR-based continuous control on Spielberg, but the two studies mature along different paths. For PPO, we iteratively refine reward shaping and observation design until the agent can make stable progress on full-track rollouts from random spawn points. For SAC, we use `f1tenth_gym` together with a short-segment curriculum, starting from easier local tasks and then scaling to longer Spielberg segments. Replay-based validation is used to verify that reported metrics match actual

driving behavior. We also run a controlled sim-to-sim study, swapping a slip-free kinematic backend for a single-track simulator with lateral slip while holding everything else fixed.

II. APPROACH

A. Common Setup

For the main comparison, both agents are evaluated on the F1 Spielberg track with a 2D LiDAR sensor as the primary observation. In addition, SAC is studied in a simulator-fidelity sub-study on the Levine track. The shared design principles are:

- **Observation:** downsampled LiDAR beams plus a small proprioceptive vector (forward speed, angular velocity / yaw rate).
- **Action:** continuous steering
- **Network:** a multilayer perceptron policy/value network. For SAC we use a 256×256 MLP via Stable-Baselines3. For PPO we use a 256×256 MLP for both the policy and value networks via Stable-Baselines3.

B. PPO Pipeline

The PPO study went through four reward and observation iterations before converging on the best architecture. The following describes the final version of the architecture and training setup.

1) *System architecture:* The PPO pipeline is built around a PyBullet-based gym. PyBullet Gym is an open-source library that provides reinforcement learning environments compatible with the OpenAI Gym interface, using the PyBullet physics engine to simulate complex autonomous racing tasks for platforms like F1TENTH. It is a wrapper that exposes the observation/action interface, the PPO learner, a validation callback, and a replay tool, as shown conceptually in Fig. 1. The gym backend was chosen specifically to bypass compatibility issues encountered in early F1TENTH-gym experiments. PyBullet gym also spawns the agent (car) at a random spot on the track to train and evaluate on. This avoids the agent overfitting on the beginning of the track.

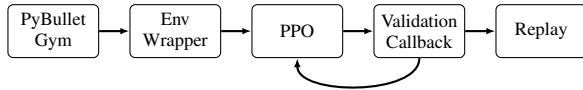


Fig. 1. PPO system architecture. The validation callback closes a metrics loop back into PPO to drive curriculum / reward refinement; the replay tool re-executes saved checkpoints with deterministic seeds.

2) *Observation Space and Action Space:* The PyBullet Gym provides all 1080 lidar beams, 6 dimensional pose information, 6 dimensional velocity information, and 6 dimensional acceleration information. However, most of these information are redundant. Most of the 1080 lidar beams contain highly redundant information and can easily be downsampled into bins to significantly decrease the observation space. This leads to a huge decrease in noise and a more robust policy. Given it is a track with no obstacles, the lidar detection does not have to be that fine-grained and can be downsampled into

less bins. The 1080 lidar beams is downsampled into 20 bins. Information like pose could lead to overfitting to a specific track, as the agent could learn the coordinates where the agents are supposed to make specific turns instead of making turns based off of the lidar observations. Since the goal was for the agent to drive based off of mainly the lidar observation, most other information are removed. We wanted to mainly focus on the steering part of driving through the race track, so there is only one action: steer. The speed is fixed at a constant speed throughout.

So the final observation and action space is the following:

- **Observation:** 20 downsampled LiDAR beams (down from the 1080 raw beams), forward speed, angular velocity.
- **Action:** Steering angle

3) *Reward design:* The overall goal for driving the car is that the car has to go through as much of the racetrack as possible without crashing. So the following is the reward function:

$$R = \text{progress_reward} + \text{collision_penalty} + \text{wrong_direction_penalty}$$

Since the driving speed is fixed, there is no reward bonus or per frame penalty to encourage faster driving.

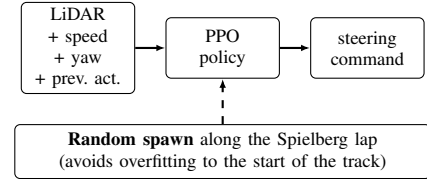


Fig. 2. PPO observation/action interface and the random-spawn curriculum used in PPO v4.

4) *Previous setup iterations:* A lot of trial-and-error happened to land on this final system setup. Here summarises the previous three iterations and explains why each of them failed/performed badly:

a) *PPO v1 — F1TENTH gym:* The agent was trained directly inside the F1TENTH gym simulator. The agent *kept driving directly into the wall*. On inspection, lighter wall grazes were sometimes not registered as collisions due to latency in the simulator, and the agent learned to exploit the small reset that wall contact produced to “slide” down the hallway instead of properly driving down. We migrated to PyBullet to eliminate this latency artefact.

b) *PPO v2 — PyBullet, dense penalties:* The PPO agent is implemented within the PyBullet gym. The reward function design is of the form

$$R = \text{per_frame_penalty} + \text{collision_penalty} + \text{progress_reward} + \text{velocity_bonus} + \text{wrong_direction_penalty}$$

with per-frame penalty -0.1 , collision $= -100$, progress $\Delta p \times 100$, velocity bonus $\text{progress} \cdot v/5 \cdot 0.5$, and wrong-direction penalty -2.0 . It is a relatively penalty heavy reward function. The agent learned a degenerate strategy: *stay alive incurs negative penalty over time, so it is cheaper to crash immediately rather than properly learning how to drive*. The per-frame penalty dominated the return.

c) *PPO v3 — sparser rewards*: Per-frame penalty was removed and reduced the collision penalty:

$$R = \text{collision_penalty} + \text{progress_reward} + \text{velocity_bonus}$$

where collision $= -10$. But then, the agent found a different exploit: *spinning in circles at the start line* to harvest small fluctuations in the progress signal between 0% and 99%.

After this, we realized a balance needs to be struck between the reward function in PPOv2 and PPOv3. All three of these setups were using the full observation and action space, which we realized later is very difficult to learn from. So, both the observation space and action space got pruned to lead to a simpler and less noisy setup, which is the final setup presented previously.

C. SAC: Algorithm and Curriculum on Spielberg

The SAC study is split into two complementary parts. The first, presented in this subsection, develops the SAC algorithm, observation / reward design, and a short-segment curriculum capable of training reproducible local policies on the Spielberg track. The second (Sec. II-D) is a controlled sim-fidelity sub-study run on the Levine track that re-uses the same SAC configuration to isolate the effect of simulator dynamics.

1) *Setup*: The goal is to train a SAC policy to drive safely and consistently on the Spielberg track using only LiDAR. The simulator is the upstream `fl1tenth_gym` together with the `fl1tenth` visualization stack; the learning algorithm is SAC from Stable-Baselines3, with an 256×256 MLP policy and SB3’s automatic entropy tuning. The policy outputs *steering only* — early experiments that coupled steering and speed at the policy output were unstable, so speed was controlled separately and the policy focused on lateral control. The system architecture mirrors the PPO pipeline of Fig. 1 but with `fl1tenth_gym` as the simulator backend and SAC as the learner (Fig. 3).

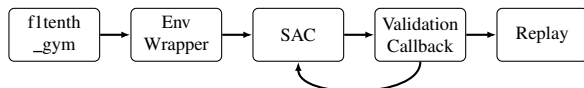


Fig. 3. SAC system architecture.

2) *Observation and reward*: The observation vector consists of downsampled LiDAR beams plus three proprioceptive scalars: forward speed, lateral speed, and yaw rate. The reward is centerline-progress-based with smoothness and safety shaping:

- **Positive** reward for centerline progress.

- **Speed** reward, gated to fire only when progress is positive (so the agent cannot earn speed reward by reversing or spinning).
- **Penalties**: steering magnitude, steering smoothness (jerk), yaw rate, track deviation, scrape, reverse, and collision.

The smoothness terms (steering magnitude, steering jerk, yaw rate) were essential to avoid policies that scored well on raw progress but exhibited oscillatory “saw-tooth” steering — replay-based inspection made it clear that without these terms, a policy could be numerically strong yet practically unusable. Track-deviation, scrape, and reverse penalties harden the policy against the geometry-exploiting failure modes seen in PPO v1–v3.

3) *Curriculum*: We use a four-stage curriculum, increasing complexity gradually (Fig. 4). The curriculum was necessary because full-track / full-lap training was too sparse to learn from scratch.

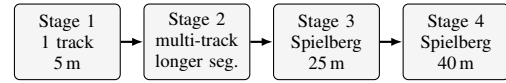


Fig. 4. SAC curriculum: short segments on a single track first, then multiple tracks, then increasing segment length on Spielberg.

4) *Training and validation pipeline*: Training proceeds in five stages (Fig. 5): **TRAIN** (collect transitions in `fl1tenth_gym`, update SAC), **VALIDATE** every 10k steps, **REPLAY** of saved checkpoints with deterministic seeds, **INSPECT** (compare reported metrics against actual driving), and **REFINE** (tighten metric logic and continue curriculum). An early “lap completion” metric was found to be too optimistic — replay-based validation against the actual rendered run was needed to catch policies that scored well numerically but drove poorly.

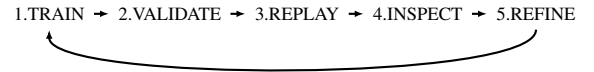


Fig. 5. SAC training and validation loop. Replay-based inspection catches policies that score well numerically but drive poorly.

D. SAC: Simulator-Fidelity Sub-Study (Levine)

The second part of the SAC study addresses one specific question: *how much does the simulator itself shape what SAC learns?* It re-uses the SAC algorithm, observation/action interface, and reward shape from the curriculum study (Sec. II-C), but applies them to a controlled comparison of two physics backends on the Levine track.

1) *Two backends behind one interface*: We wrap two physics backends behind one identical Gymnasium interface (`Fl1tenthRLEnv`): **Exp 1** a custom kinematic-bicycle simulator (no slip, fast, no `fl10` dependencies), and **Exp 2** the upstream `fl10_gym` (RK4 single-track dynamics with lateral slip and raycast LiDAR over the track image). Observation, action, reward shape, network, and SAC hyperparameters are

identical across both backends. Training budget is 200k env-steps (~ 20 min CPU) on each. Because both experiments share the same wrapper, a policy trained on one backend can be loaded into the other with no code changes.

III. RESULTS

A. PPO Results

After four reward iterations and observation simplification, PPO learns a non-trivial driving policy on Spielberg:

- The agent drives **down straights** reliably.
- The agent makes **most corners**, with smoother (longer radius) corners taken more consistently than sharp corners.
- Because the agent is trained from **random spawn points**, it does not overfit to the start of the track and can recover from mid-track positions.

The dominant lessons from the PPO experiments were practical:

- **Simplifying and normalising observations helped a lot.** Cutting LiDAR from 1080 to 20 beams and removing pose / most velocity features sharply reduced noise and overfitting.
- **A velocity bonus is more effective than a frame penalty.** Frame penalties cause the agent to suicide before it has time to explore; velocity bonuses encourage the desired behaviour without that failure mode.
- **Numbers don't tell the full story.** Reward exploitation (spinning, wall-grazing) can produce high reward with bad behaviour. We treat visualisation / replay as a first-class evaluation tool, not as an afterthought.
- **Reward tuning is the dominant cost.** It took many iterations and significant manual diagnosis to land on a non-degenerate configuration.

Fig. 6 shows six representative frames from the PPO rollout video on Spielberg, illustrating the learned driving behaviour (straight-line tracking, smooth-corner entry, and recovery from off-centre positions).

Table I shows 10 episodes of the trained PPO agent driving down the Spielberg race track. It shows the progress gained driving down the race track, the reward achieved, and the number of timesteps the agent was alive in the episode. Table ?? shows 10 episodes of the trained PPO agent driving down a 40m segment of the Spielberg race track. To keep things consistent to compare with SAC, we ran evaluation on the same 40m segment of the Spielberg track.

B. SAC Results — Spielberg curriculum study

1) *Curriculum scaling*: Early curriculum experiments showed that scaling segment length and the number of tracks was possible, but not every branch was stable. Some configurations produced policies that looked reasonable in training-time reward but collapsed in replay. This was the original motivation for the TRAIN $\rightarrow$ VALIDATE $\rightarrow$ REPLAY $\rightarrow$ INSPECT $\rightarrow$ REFINE loop (Fig. 5).

Episode	Progress gained	Reward
1	24.3%	14.29
2	18.5%	8.50
3	18.0%	7.99
4	23.9%	13.91
5	18.4%	8.38
6	20.0%	9.95
7	23.5%	13.47
8	18.3%	8.31
9	18.5%	8.50
10	22.7%	12.66
Mean	20.6%	10.60

TABLE I

10 EVALUATION EPISODES ON THE FULL TRACK WITH A TRAINED PPO AGENT

Episode	Progress gained	Reward
1	100.0%	15.1
2	20.7%	2.9
3	51.4%	5.8
4	100.0%	15.1
5	41.3%	4.8
6	32.7%	3.3
7	56.9%	6.2
8	100.0%	15.0
9	47.2%	5.3
10	100.0%	15.2
Mean	65.2%	8.9

TABLE II

10-EPISODE EVALUATION TABLE FOR THE TRAINED PPO AGENT ON THE 40 M SPIELBERG SEGMENT TASK.

The strongest SAC path was a Spielberg-focused curriculum:

- a short single-track stage,
- a broader multi-track stage to expose the policy to more track geometries,
- a 25 m Spielberg segment stage,
- and a 40 m Spielberg segment stage initialised from the 25 m policy.

This curriculum produced clear improvements in local driving behavior as segment length increased from 25 m to 40 m. However, the SAC study was not carried all the way to a stable full-track benchmark comparable to the PPO evaluation, so the curriculum results should be interpreted as evidence of local-policy scaling, and this might explain the faster convergence and improved rewards

2) What worked:

- **Multi-track curriculum** (Stage 2) helped identify useful settings, because exposing the policy to centerlines from multiple tracks gave a broader set of driving geometries and prevented overfit to the geometry of a single track.
- **Narrowing back to Spielberg** (Stages 3–4) gave the strongest *replay consistency* on the target track — a single-track specialist is more reliable on that track than a generalist policy trained jointly across many tracks.
- Together, the broaden-then-narrow recipe is a **combined workflow for scaling the RL agent**: broaden to learn structure, then narrow to harden the target.

3) *Driving behaviour*: On Spielberg the trained agent drives down straights well, makes most corners, and is more

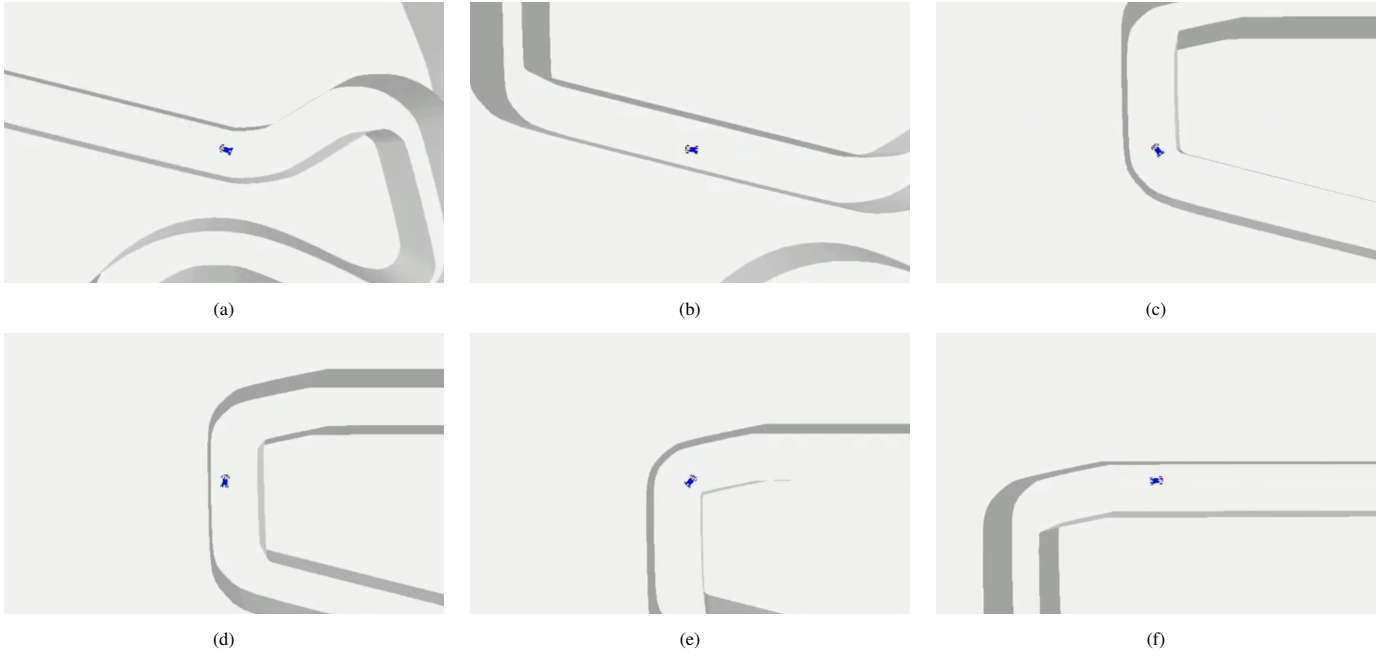


Fig. 6. Six representative frames from the PPO rollout video on Spielberg. (a)–(b) straight-line tracking; (c)–(d) corner entry and apex; (e)–(f) recovery to centre after a wider racing line.

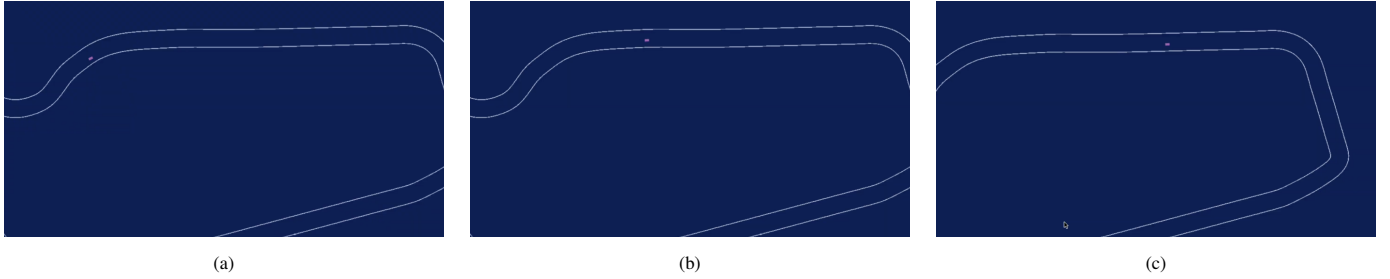


Fig. 7. Three representative frames from the SAC rollout video on Spielberg. (a) straight-line tracking near the centerline; (b) smooth-corner entry; (c) mid-track recovery from a non-start spawn, demonstrating that the curriculum prevents start-position overfit.

consistent on smoother (longer-radius) turns than on sharp turns. Because training spans random spawn points across multiple curriculum stages, the agent does *not* overfit to the start of the track and can recover from mid-track spawns. Fig. 7 shows three representative frames from the SAC rollout video on Spielberg.

4) *Lessons specific to the algorithmic study:* The Spielberg curriculum study produced a small set of algorithm-and-design lessons (separate from the cross-sim findings of the next sub-study):

- **Steering-only beats joint steering+speed** as a starting point; coupling them invites instability that a curriculum alone cannot fix.
- **Lap completion as a single metric is too optimistic;** replay-based inspection is required to catch numerically-strong but practically-unusable policies.
- **Auto-tuned entropy** (α) avoided the collapse / non-convergence failure modes typically associated with SAC’s entropy coefficient.

Episode	Progress gained	Reward
1	100.0%	15.18
2	58.4%	5.94
3	42.1%	4.07
4	100.0%	15.34
5	61.0%	6.22
6	56.3%	5.73
7	100.0%	15.09
8	39.7%	3.81
9	59.2%	6.01
10	100.0%	15.27
Mean	71.7%	9.37

TABLE III

REPRESENTATIVE 10-EPIISODE EVALUATION TABLE FOR THE TRAINED SAC AGENT ON THE 40 M SPIELBERG SEGMENT TASK.

Table IV compares PPO and SAC on the same 40 m Spielberg segment task. Under this matched local evaluation, SAC achieved slightly higher mean progress and mean reward than PPO. This difference is likely due in part to the curriculum structure used for SAC, which progressively scaled the task from easier short-horizon segments to harder local objectives.

A plausible explanation is that this setup aligns especially well with SAC’s off-policy nature, since the replay buffer allows experience from earlier easier stages to remain useful during later training. PPO, by contrast, is on-policy and relies primarily on newly collected rollouts, making it more sensitive to the current task distribution and therefore more dependent on stable reward shaping and exploration.

RL agent	Mean Progress Gained	Mean Reward
PPO	65.2%	8.9
SAC	71.7%	9.37

TABLE IV

COMPARISON OF MEAN METRICS BETWEEN PPO AND SAC AGENTS ON THE 40 M SPIELBERG SEGMENT TASK.

C. SAC Results — Simulator-Fidelity Sub-Study (Levine)

The remainder of the SAC results report the sub-study that compares two physics backends on the Levine track using the same SAC configuration as above.

1) *Training behaviour and convergence*: On both backends, the same SAC configuration converges under a 20-minute training budget. On the kinematic sim (Fig. 8), evaluation reward stabilises at roughly 18 after ~ 150 k steps; on `f110_gym` (Fig. 9), it stabilises around 120. The reward magnitudes are not directly comparable because episode lengths and reward scales differ per backend; what matters is that *both* curves converge cleanly. This means the SAC config is sufficient on either simulator, and any downstream gap we measure between them is driven by *dynamics*, not by training instability.

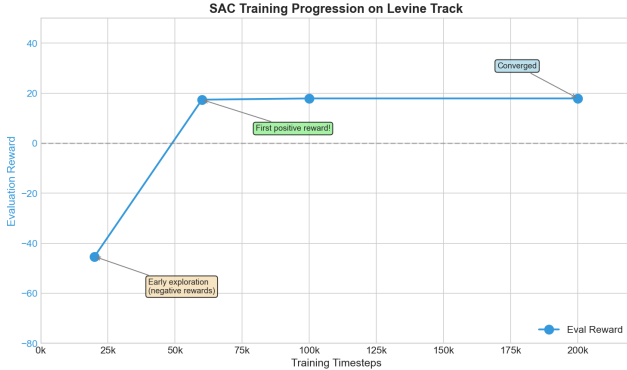


Fig. 8. SAC training progression on the kinematic-bicycle simulator (Levine track). Reward stabilises around 18 after ~ 150 k steps.

2) *Quantifying the simulator gap*: The single most informative result of the SAC study is the cross-simulator evaluation. We load the policy trained in the kinematic sim into `f110_gym` with no code changes and evaluate it on the same Levine track:

The kinematic-trained policy collapses to -51 reward on `f110_gym` versus $+120$ for the policy that was trained there directly: a ~ 170 -point gap (Fig. 10). Episodes terminate at ~ 191 steps versus 762. The cause is physical: the kinematic model never had to learn slip, so the moment lateral dynamics

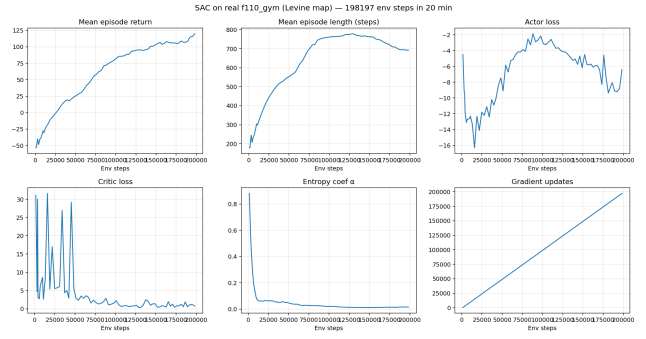


Fig. 9. SAC training progression on the upstream `f110_gym` backend (RK4 single-track dynamics, Levine track).

TABLE V
CROSS-SIMULATOR EVALUATION OF SAC POLICIES.

Train $\rightarrow$ Test	Return	Ep len	Crash
kin $\rightarrow$ kin (200 ep)	17.9 ± 0.1	491	100% (timeout)
f110 $\rightarrow$ f110 (20 ep)	120 ± 168	762	80%
kin $\rightarrow$ f110 (10 ep)	-51 ± 12	191	100%

show up, the action distribution saturates at the steering clip and forward speed stalls.

3) *Combined SAC takeaways*: Across both the Spielberg curriculum study and the Levine sim-fidelity sub-study, the SAC iterations produced a small set of practical lessons (Table VI). The first three rows come primarily from the curriculum study; the last two come from the sub-study.

TABLE VI
SAC LESSONS LEARNED, ACROSS BOTH SAC STUDIES.

Area	Failure	Lesson
Control output	Steering + speed unstable	Start steering-only
Horizon	Full laps too hard	Short-segment curriculum
Metrics	Lap completion too optimistic	Validate with replay
Baselines	Pure Pursuit was brittle	Tune & test more carefully
Sim fidelity	Kin. policy fails on f110	Train on deploy dynamics

IV. DISCUSSION

A. On-policy vs. off-policy in this domain

The experimental results demonstrate that both Proximal Policy Optimization (PPO) and Soft Actor-Critic (SAC) are capable of learning useful end-to-end driving behavior for the F1TENTH platform, but they matured under different task formulations. PPO reached a mean progress gain of 20.6% on full-track Spielberg rollouts with random spawn points. Under the matched 40 m segment evaluation, PPO achieved a mean progress gain of 65.2% and mean reward of 8.9, while SAC

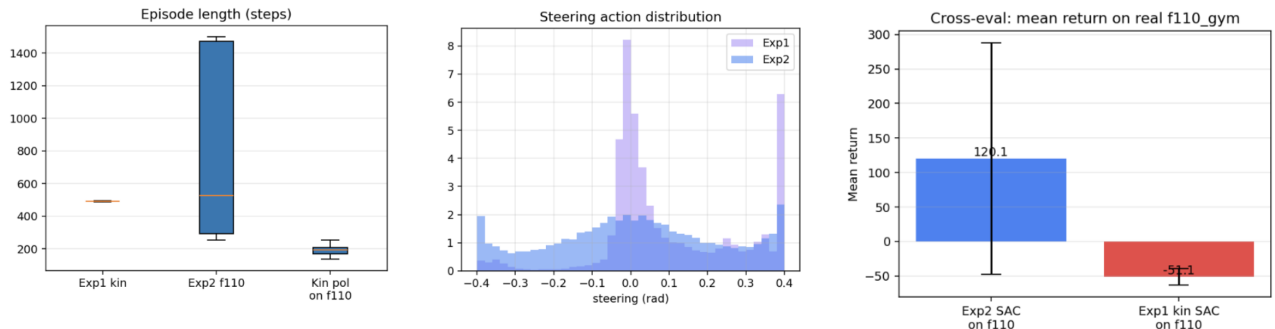


Fig. 10. Cross-simulator evaluation. Left: episode-length distribution (kinematic-trained policy survives ~ 191 steps on `f110_gym` vs ~ 762 for the natively trained policy). Centre: steering action distribution (kinematic-trained saturates at the clip on `f110`). Right: mean return on `f110_gym` for the two policies — the kinematic-trained policy collapses while the `f110`-trained policy holds.

achieved 71.7% mean progress and 9.37 mean reward. SAC also reached best validation clean completion rates of 1.00 on both 25 m and 40 m Spielberg segment tasks, suggesting that the curriculum was effective at improving local driving behavior as horizon length increased.

The primary distinction observed was the stability and sample efficiency of the two methods. PPO, being an on-policy method, required careful management of the observation space to avoid the “catastrophic forgetting” or divergence often seen when training on high-dimensional raw LiDAR data. SAC, utilizing off-policy transitions and maximum entropy reinforcement learning, benefited significantly from the curriculum, allowing it to reuse past experiences to refine its lateral control.

Both studies concluded that “less is more” regarding sensor input. Reducing LiDAR input from 1080 to 20 beams was a turning point for PPO, as it stripped away redundant noise that otherwise led to overfitting.

One plausible explanation for the different training behavior is the on-policy versus off-policy distinction. SAC, as an off-policy method, can reuse transitions collected during earlier and easier curriculum stages through its replay buffer. This may make curriculum scaling particularly effective, since short-segment experience can continue to support learning as the horizon increases. PPO, by contrast, is on-policy and depends mainly on newly collected trajectories from the current policy, which may make it more sensitive to reward design, observation noise, and unstable exploration during training.

B. Challenges encountered

- **Entropy sensitivity (SAC).** Wrong α values cause either policy collapse or non-convergence. We mitigated this by using SB3’s automatic entropy tuning, which worked without issue.
- **Reward exploitation.** The agents repeatedly found unintended shortcuts (wall-grazing, spinning, suiciding). The fix was not to penalise more aggressively but to remove the exploitable gradient (e.g. remove per-frame penalty in PPO; use replay-based validation in SAC).

- **Environment compatibility.** Different gyms expose different APIs and ship with conflicting Python pin-versions. We worked around this by isolating each backend behind `FltenthRL`Env and using a separate Python 3.10 environment for the legacy `f110_gym==0.2.1` dependency.
- **Training speed and compute.** Training is wall-clock-bound on CPU; longer or larger studies require renting GPU / cloud compute.

V. LIMITATIONS AND FUTURE WORK

Several limitations remain in this study. First, not all PPO and SAC results were obtained under perfectly matched evaluation settings: PPO was studied both on full-track Spielberg rollouts and on a matched 40 m segment task, whereas SAC showed its strongest performance in a short-segment curriculum setting. As a result, the 40 m comparison is useful for local-policy evaluation, but it is not yet a complete full-track direct benchmark. Second, both agents still struggled with sharp corners and longer-horizon robustness, especially when evaluation conditions became more demanding. Third, the SAC results highlighted a simulator-fidelity issue: policies that perform well in a simplified simulator may not transfer cleanly to a higher-fidelity dynamics model.

Future work should proceed in three directions. The first is to build a more directly comparable benchmark between PPO and SAC by evaluating both methods on the same track, the same horizon, and the same success metric. The second is to extend the SAC curriculum further, gradually scaling from short local segments toward longer track sections and eventually full-track driving. A plausible hypothesis is that SAC could approach stronger full-track performance if given additional curriculum stages and training time, since its segment-level performance improved as horizon length increased. The third is to re-introduce longitudinal control through a more careful curriculum, so that the policy learns not only steering but also braking and acceleration for sharper corners and more realistic racing lines. A final long-term goal is true sim-to-real validation on physical FITENTH hardware,

where both reward design and simulator assumptions can be tested against real vehicle behavior.

VI. CONCLUSION

We presented a head-to-head investigation of on-policy (PPO) and off-policy (SAC) deep reinforcement learning for end-to-end LiDAR-based control on F1TENTH. The two methods showed different strengths and different bottlenecks. PPO required several iterations of reward shaping and a heavily simplified observation space before producing a policy that could make steady progress around the full Spielberg track, reaching a mean progress gain of 20.6% under random-spawn evaluation. On the matched 40 m segment task, PPO achieved 65.2% mean progress and 8.9 mean reward.

SAC followed a different path. Rather than immediately scaling to a comparable full-track benchmark, SAC benefited from a curriculum of shorter local tasks, reaching best validation clean completion rates of 1.00 on both 25 m and 40 m Spielberg segment stages. On the matched 40 m segment evaluation, SAC achieved 71.7% mean progress and 9.37 mean reward, suggesting that curriculum scaling aligned well with SAC’s off-policy replay-based learning. In a separate controlled simulator study on Levine, SAC reached 74% of Pure Pursuit’s mean reward in the kinematic environment using LiDAR alone, showing that an off-policy agent can learn competitive local driving behavior without privileged waypoint-following inputs. However, this same study also revealed SAC’s major weakness: sensitivity to simulator fidelity. A policy trained in the simplified kinematic simulator transferred poorly to the higher-fidelity `f110_gym` dynamics model, indicating that dynamics mismatch can dominate performance even when the learning algorithm remains unchanged.

Overall, our results suggest that PPO and SAC are limited by different factors: PPO is more sensitive to reward design, whereas SAC is more sensitive to curriculum design and simulator dynamics. Taken together, these findings support the view that deep reinforcement learning is a viable alternative to classical control on the F1TENTH platform, but only when reward definitions, simulator assumptions, and evaluation metrics are carefully aligned with the behavior one actually wants to deploy.

REFERENCES

- [1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv:1707.06347*, 2017.
- [2] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” in *ICML*, 2018.
- [3] T. Haarnoja *et al.*, “Soft actor-critic algorithms and applications,” *arXiv:1812.05905*, 2018.
- [4] M. O’Kelly, H. Zheng, D. Karthik, and R. Mangharam, “F1TENTH: An open-source evaluation environment for continuous control and reinforcement learning,” in *NeurIPS PMLR*, 2020.
- [5] F1TENTH project, *f1tenth_gym*: open racing simulator. https://github.com/f1tenth/f1tenth_gym
- [6] A. Raffin *et al.*, “Stable-Baselines3: Reliable reinforcement learning implementations,” *JMLR*, 2021.
- [7] A. Brunnbauer *et al.*, “Latent imagination facilitates zero-shot transfer in autonomous racing,” *ICRA*, 2022.